KIIT TRANSIT

Smart Campus Transportation System

Platform	Android (Primary) Web Admin (Optional)
Tech Stack	Flutter • Supabase • Flutter Map (OpenStreetMap)
Architecture	MVVM with Repository Pattern
Figma Link:	https://www.figma.com/design/Q2EVvOhWUOCF5L0NEwlE63/Untitled?node-id=0-1&t=X4S86aC0CMzi9qOn-1

1. Executive Summary

KIIT Transit is a campus-wide smart transportation platform designed to solve a fundamental problem: students at KIIT have zero visibility into the real-time location and status of the campus EV carts and buses. Today, students stand at pickup points and wait — without knowing whether the next vehicle is 2 minutes away or has already passed.

The system consists of two interconnected mobile applications: a Driver App that passively broadcasts GPS location while on duty, and a Student App that displays all active vehicles live on an interactive campus map — exactly like Ola or Uber, but built for the KIIT ecosystem. Together, they turn the current fragmented, opaque transport network into a visible, predictable, and efficient system.

The goal of Version 1 is deliberately focused: answer the single most important question students have — "Where are the buses and EVs right now?" — and do it reliably. Features like ride booking, passenger counting, and ML-based prediction are intentionally deferred to later phases so that the core system can be built, tested, and deployed quickly.

2. Problem Statement

KIIT's campus spans multiple academic blocks, hostels, food courts, and gates, with EV carts and buses serving as the primary means of getting around. Despite the infrastructure existing, the student experience of using it is deeply frustrating. The core issue is not a lack of vehicles — it is a complete lack of information.

2.1 No Real-Time Visibility

Students have no way to find out where a bus or EV is at any given moment. There is no application, website, or display board that communicates vehicle locations. Students standing near Campus 14, for example, cannot know whether the nearest EV is 300 metres away and approaching, or has just departed and will not return for 20 minutes.

2.2 Uninformed Waiting

Because there is no information, students wait blindly at pickup points. This results in extended, uncertain waits that could be avoided if the student had even basic ETA data. A student who knows the bus is 12 minutes away can go get food first. A student who knows the EV is 90 seconds away stays put. Without information, both students stand and wait indefinitely.

2.3 No Centralized Schedule Platform

Bus schedules exist, but students must memorize them or rely on informal word-of-mouth. There is no digital, always-accessible schedule system. Day scholars in particular are frequently caught off-guard by schedule changes, delays, or missed last buses, since there is no platform to notify them or let them check in real time.

2.4 Fragmented, Siloed Operations

EV carts and buses operate as completely independent systems. A student wanting to reach their hostel from an academic block must mentally track two separate, invisible networks. There is no unified interface that shows all available transport options in a single view, forcing students to guess and hope.

> **The Core Problem in One Sentence**
> Students at KIIT have zero visibility into their own campus transportation network, causing unnecessary waiting,
> missed vehicles, and a generally inefficient commuting experience that affects thousands of students daily.

3. Proposed Solution

KIIT Transit builds a real-time transportation visibility layer on top of the existing campus infrastructure. No hardware changes are required. No new vehicles are needed. The system leverages the smartphones already carried by drivers to broadcast GPS locations, and the smartphones already carried by students to receive and display them.

The driver's phone becomes the live data source. The student's phone becomes the information display. Supabase Realtime acts as the bridge, propagating location updates from driver devices to every student watching the map in near-real time. The result is a campus-scale, Ola-style vehicle tracking experience built with lightweight, open-source tooling.

3.1 What the System Will Provide

- Live map showing all active buses and EV carts on campus at the same time
- Automatic ETA calculation based on vehicle distance and estimated speed
- Route trails showing which direction a vehicle is heading and which stops it serves
- Digitized bus schedules with real-time delay or on-time status
- High-demand request system allowing students to signal their location to nearby drivers
- Driver contact directory for situations requiring direct communication

3.2 What the System Will Not Do (Version 1)

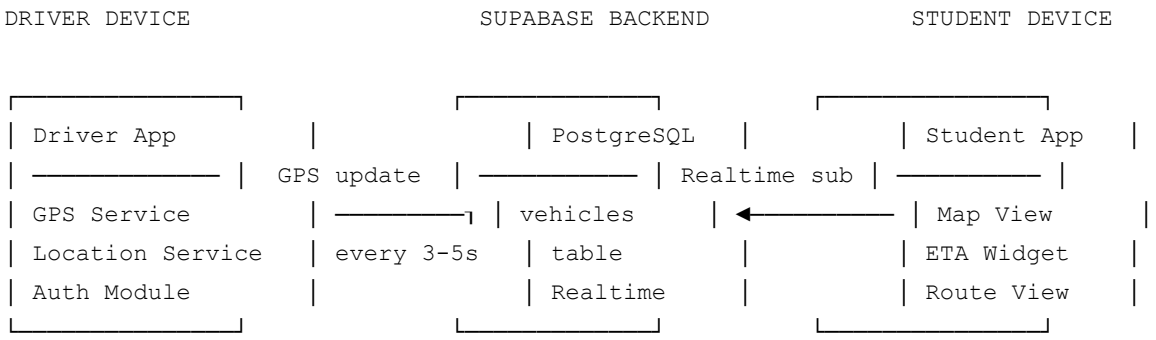
To keep development feasible and the driver experience frictionless, the following are explicitly excluded from Version 1:

- Ride booking or personal vehicle reservation
- Passenger count tracking (requires driver manual input or complex automation)
- Push notifications (planned for Version 2 via FCM or OneSignal)
- ML-based ETA prediction (planned for Version 3)

4. System Architecture

The platform is built around two distinct user-facing applications — the Student App and the Driver App — both sharing a common Supabase backend. The architecture follows a clean MVVM (Model-View-ViewModel) pattern, ensuring clear separation of concerns and making the codebase easy to extend as new features are added.

4.1 High-Level Data Flow



4.2 User Roles

Role	Access Level	Primary Responsibility
Student	Read-only map, ETA, schedules, request submission	Consume transportation data to plan their commute
Driver	GPS broadcast, duty status, demand request visibility	Share real-time location; view demand signals from students
Admin	Full system access via admin dashboard (Phase 2)	Monitor fleet, manage schedules, review analytics

5. Core Features (Version 1)

5.1 Real-Time Vehicle Tracking

The centrepiece of KIIT Transit. While a driver is on duty, their device continuously sends GPS coordinates to Supabase every 3–5 seconds. Every student app subscribed via Supabase Realtime receives these updates instantly and animates the corresponding vehicle marker to its new position on the map.

The result is a live, moving map of every active bus and EV on campus. A student can open the app from their hostel room, see a vehicle 400 metres away heading in their direction, and time their departure accordingly. The core student experience is identical to opening Ola — but the vehicles are campus EVs and buses.

Vehicle markers will be visually distinct by type (bus vs. EV cart) and will include a directional arrow indicating heading, so students can immediately tell whether a vehicle is approaching or moving away.

5.2 Estimated Arrival Time (ETA)

The app calculates a simple, practical ETA for each vehicle relative to the student's current position. Version 1 uses a straightforward formula that is accurate enough to be genuinely useful without the overhead of ML modelling:

```
ETA (minutes) = Distance (metres) / Estimated Speed (metres per minute)

Example:
  Vehicle EV-03 is 650m from student
  Estimated campus speed: ~250 m/min (15 km/h)
  ETA = 650 / 250 = 2.6 minutes ≈ 3 minutes
```

ETAs are recalculated automatically as the vehicle moves. This alone eliminates the most frustrating part of campus transport: students no longer have to guess whether it is worth waiting or walking.

5.3 Interactive Campus Map

Built with Flutter Map and OpenStreetMap tiles, the campus map serves as the primary interface for both students and drivers. The map is pre-configured to centre on KIIT's campus and includes named markers for key landmarks:

- Academic blocks (Campus 6, Campus 14, Campus 15, KP-1 through KP-15)

- Hostels and student housing areas
- Library, food courts, and common facilities
- Main gates and day scholar pickup zones

These landmarks give students spatial context, making it easy to interpret vehicle positions at a glance. Importantly, OpenStreetMap is free, open-source, and requires no API key or billing setup, making it the practical choice for a student project that needs reliable, long-term operation.

5.4 Route Visualization

Fixed routes (for scheduled buses and route-based EVs) are drawn as polylines on the map. Students can tap a vehicle to see its full planned route, including upcoming stops in sequence. This helps students determine whether a bus serves their destination before they commit to waiting for it.

For buses, the route also serves as a wayfinding tool — students new to campus can use it to understand the transport network at a glance.

5.5 Schedule Management

Bus schedules are digitized within the application. Rather than memorizing departure times or relying on notice boards, students can open the Schedule view and see:

- The next scheduled departure for each bus route
- Current status: On Time, Delayed, or Cancelled
- Estimated delay in minutes if the bus is running late
- Next 3–5 upcoming departures for each route

For day scholars who depend on specific buses to reach campus or return home, this feature alone significantly reduces stress and missed connections.

5.6 High-Demand Request System

This feature addresses a common scenario: a large group of students is stranded at a location with no vehicle in sight. Instead of individual students waiting helplessly, any student can submit a demand request for their current location. The request is aggregated and displayed to nearby drivers as a demand signal.

This is not ride booking. There is no confirmation, no guaranteed pickup, and no obligation on the driver. It is simply a demand visibility tool — if 12 students are waiting at Campus 14, nearby drivers can see that and choose to serve the location. The feature requires no changes to how drivers operate; it only adds information they currently have no access to.

5.7 Driver Directory

The app includes an authorized directory of drivers, their assigned vehicles, and contact numbers, accessible to students when needed. Access to contact numbers may be restricted to Admin or made universally visible, depending on administrative preference. This is particularly useful in situations where a student needs to flag down a specific driver or report a vehicle issue.

6. Technology Stack

6.1 Frontend — Flutter

Framework	Flutter (Dart)
State Management	Provider or Riverpod (ViewModel layer)
Navigation	GoRouter for declarative, role-aware routing
Maps	Flutter Map package with OpenStreetMap tile layer
Location	geolocator + location packages for GPS access
Realtime	supabase_flutter SDK for database subscriptions

6.2 Backend — Supabase

Supabase provides the entire backend without requiring a custom server. It combines a PostgreSQL database, authentication, realtime subscriptions, and row-level security in a single managed platform. The free tier is more than sufficient for a campus-scale deployment, and the system remains open-source with no vendor lock-in.

Service	Supabase (self-hostable, open-source)
Database	PostgreSQL — structured relational storage for all data
Authentication	Email + password and Google auth with college email domain restriction
Realtime	Supabase Realtime — WebSocket-based live DB subscriptions
Row-Level Security	Postgres RLS policies for role-based data access
Hosting	Supabase free tier (up to 500MB DB, 2GB bandwidth/month)

6.3 Maps — Flutter Map + OpenStreetMap

OpenStreetMap is a free, open-source map provider that does not require API keys or billing configuration. Flutter Map is a Leaflet-inspired Flutter package that renders OSM tiles and supports custom markers, polyline overlays, circles, popups, and marker clustering. The map is

purely a visual canvas — vehicle tracking is handled entirely by Supabase Realtime, not by the map provider.

6.4 Authentication Design

Authentication uses Supabase Auth with email and password. The system restricts registration to KIIT institutional email addresses (kiit.ac.in domain) to prevent unauthorized access. On successful login, the user's role (student, driver, or admin) is read from the profiles table, and the app routes them to the appropriate interface automatically.

7. Database Design

The database schema is designed to be minimal for Version 1 — only the tables and fields necessary for the MVP are included. Future features (analytics, notifications, QR boarding) will add to this schema without requiring structural changes to the core tables.

7.1 Schema Overview

```
TABLE: profiles
├─ id              UUID          PK, FK → auth.users.id
├─ name            TEXT          Student or driver full name
├─ email           TEXT          Institutional email address
├─ role            TEXT          'student' | 'driver' | 'admin'
└─ created_at      TIMESTAMPTZ   Account creation timestamp

TABLE: vehicles
├─ id              UUID          PK
├─ vehicle_number  TEXT          e.g. 'EV-03', 'BUS-01'
├─ vehicle_type    TEXT          'ev' | 'bus'
├─ driver_id       UUID          FK → profiles.id
├─ latitude        FLOAT8        Current GPS latitude
├─ longitude       FLOAT8        Current GPS longitude
├─ heading         FLOAT4        Direction of travel (0–360°)
├─ status          TEXT          'active' | 'idle' | 'offline'
└─ last_updated    TIMESTAMPTZ   Timestamp of last GPS update

TABLE: routes
├─ id              UUID          PK
├─ route_name      TEXT          e.g. 'Route A — Hostel to KP-15'
└─ stops           JSONB         Ordered array of stop names and coordinates

TABLE: schedules
├─ id              UUID          PK
├─ route_id        UUID          FK → routes.id
├─ departure_time  TIME          Scheduled departure
├─ arrival_time    TIME          Expected arrival at final stop
└─ delay_minutes   INT           Live delay (0 if on time; updated by driver or
admin)

TABLE: requests
├─ id              UUID          PK
```

```
├─ location_name    TEXT          Named campus location (e.g. 'Campus 14 Gate')
├─ latitude         FLOAT8        Request GPS latitude
├─ longitude        FLOAT8        Request GPS longitude
├─ student_count    INT           Aggregated number of waiting students
└─ created_at       TIMESTAMPTZ   Timestamp of first request at this location
```

8. Project Folder Structure (MVVM)

The project follows the MVVM (Model-View-ViewModel) pattern with a clean separation between data, business logic, and UI. The structure below is the proposed starting point and will evolve during development.

```
lib/
├── core/                          # Shared infrastructure
│   ├── constants/                   # App-wide constants (routes, keys, API
URLs)
│   ├── theme/                       # Colors, typography, and theme config
│   ├── utils/                       # Utility functions (ETA calc, distance,
formatting)
│   ├── services/
│   │   ├── auth_service.dart        # Supabase auth wrapper (login, signup,
signout)
│   │   ├── location_service.dart    # Device GPS access using geolocator
│   │   ├── realtime_service.dart    # Supabase Realtime subscription management
│   │   └── map_service.dart         # Map utilities: distance calc, marker
helpers
│   └── routing/                     # GoRouter configuration, role-based guards
│
├── data/                          # Data layer (models, repositories, sources)
│   ├── models/
│   │   ├── user_model.dart
│   │   ├── vehicle_model.dart       # Includes lat, lng, heading, status, ETA
│   │   ├── route_model.dart
│   │   ├── request_model.dart
│   │   └── schedule_model.dart
│   ├── repositories/              # Business logic: abstracts data sources
│   │   ├── auth_repository.dart
│   │   ├── vehicle_repository.dart
│   │   ├── route_repository.dart
│   │   └── request_repository.dart
│   └── datasources/
│       ├── supabase_source.dart     # All Supabase read/write/subscribe calls
│       └── location_source.dart     # Raw GPS stream from device
│
```

```
  ├── viewmodels/                    # Business logic exposed to UI via
ChangeNotifier
  │   ├── auth_viewmodel.dart
  │   ├── map_viewmodel.dart          # Manages live vehicle list, map state
  │   ├── vehicle_viewmodel.dart      # Driver: duty start/stop, GPS broadcast
  │   ├── route_viewmodel.dart
  │   └── request_viewmodel.dart
  │
  ├── views/                         # UI layer
  │   ├── auth/
  │   │   ├── login_page.dart
  │   │   └── signup_page.dart
  │   ├── student/
  │   │   ├── home_page.dart          # Entry point: map with live vehicles
  │   │   ├── map_page.dart           # Full-screen interactive map
  │   │   ├── schedule_page.dart      # Bus schedule list with delay status
  │   │   └── request_page.dart       # High-demand request submission
  │   ├── driver/
  │   │   ├── dashboard_page.dart     # Driver home: duty status, demand signals
  │   │   ├── vehicle_selection_page.dart
  │   │   └── tracking_page.dart      # Active tracking UI with start/stop
controls
  │   └── common/
  │       ├── splash_page.dart
  │       └── profile_page.dart
  │
  ├── widgets/                       # Reusable UI components
  │   ├── vehicle_marker.dart         # Animated map marker (bus/EV icon +
direction arrow)
  │   ├── route_card.dart             # Route list item with stop preview
  │   ├── request_card.dart           # Demand signal card for driver dashboard
  │   └── eta_widget.dart             # ETA display badge (e.g. '3 min away')
  │
  └── main.dart                      # App entry point, Supabase init, theme,
routing
```

9. Development Phases & Roadmap

Phase 1 — MVP (Version 1.0)

The MVP is focused entirely on delivering the core value proposition: real-time vehicle visibility. Everything else is deferred.

Feature	Details
User authentication	Email + password, KIIT domain restriction, role assignment
Driver GPS broadcast	Background location stream → Supabase vehicles table every 3–5s
Live vehicle map	Student app receives Realtime updates; markers animate smoothly
Vehicle type markers	Distinct icons for bus vs. EV cart, with heading arrow
ETA calculation	Distance / speed formula for each visible vehicle
Route polylines	Pre-loaded fixed routes drawn on map with named stops
Schedule view	Digitized bus timetables with on-time / delayed status
Demand request	Student submits location need; aggregated count shown to drivers
Driver directory	Basic contact info for authorized transport personnel

Phase 2 — Enhanced Experience (Version 2.0)

- Push notifications via Firebase Cloud Messaging (FCM) or OneSignal
- Bus arriving at nearby stop — automatic alert when ETA drops below threshold
- Service delay announcements from admin or drivers
- QR-based boarding for automatic passenger count tracking
- Vehicle occupancy display (seats remaining) on student map view

- Admin dashboard: active fleet map, route utilization, demand heatmap
- Offline schedule caching so students can view timetables without connectivity

Phase 3 — Intelligence Layer (Version 3.0)

- ML-based ETA prediction using historical travel data and time-of-day patterns
- Campus demand heatmaps showing peak waiting zones by time of day
- Transportation analytics: busiest routes, average wait times, utilization rates
- Smart demand prediction: system suggests where to position idle EVs proactively
- Emergency SOS button: students can broadcast location + alert to admin while in transit
- iOS application release (dual-platform coverage)

In Summary

KIIT Transit turns a blind, fragmented campus transport system into a visible, intelligent, and predictable one — using tools the team already knows, infrastructure that already exists, and a scope that can realistically ship in a few weeks of focused development.